

Quantum Convolutional Neural Network for Image Classification using Tensorflow



Imam Muhajir · [Follow](#)

Published in Analytics Vidhya

7 min read · Oct 4, 2021



Listen



Share

Implementation Quantum in Neural Network

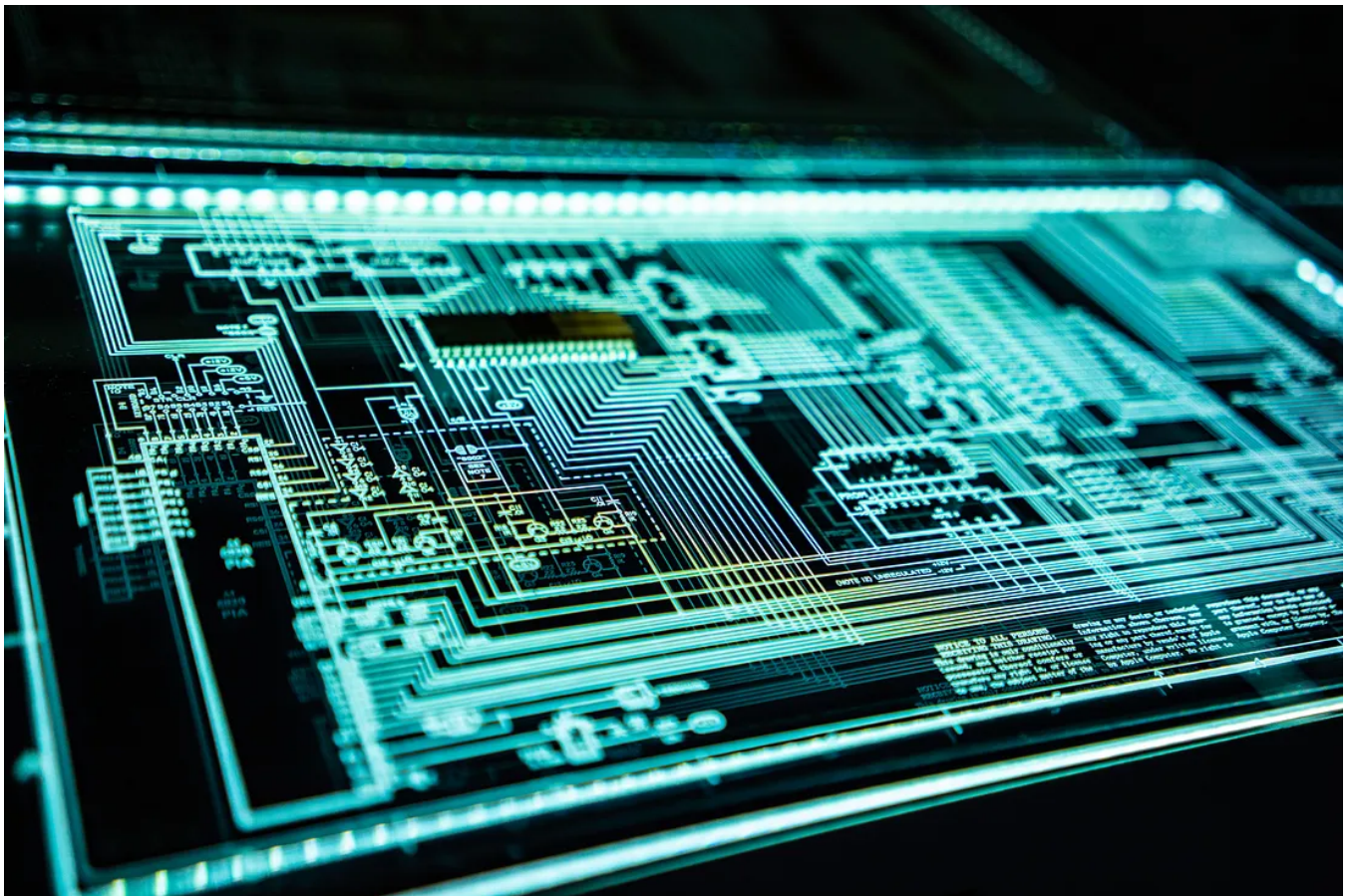


Photo by [Adi Goldstein](#) on [Unsplash](#)

INTRODUCTION

Quantum computing has the potential to outperform classical computers and is expected to play an active role in various fields, in quantum machine learning,

quantum computing has been found to be useful for enhanced feature representation and state approximation (Watabe et al. 2021). Quantum computing based on quantum bits (qubits) is inspired by the phenomenon of particles in quantum states. This phenomenon is a superposition phenomenon where a particle in a quantum state can have two states at once (Gultom 2017). The approach to this phenomenon is transformed into computations in the form of qubits, namely 0.1 or both (Kaye 2007).

MAIN COMPONENT

QUBIT

Quantum computing uses qubits as the basic unit of information, which is represented by a state vector (Gado and Younes, 2021). Dirac notation is one of the notations used in quantum mechanics which was discovered by Paul Dirac. In Dirac notation the symbol that identifies a vector is written as:

$$|a\rangle = \text{ket}$$

$$\langle a| = \text{bra}$$

example

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\langle 0| = [1 \quad 0]$$

$$\langle 1| = [0 \quad 1]$$

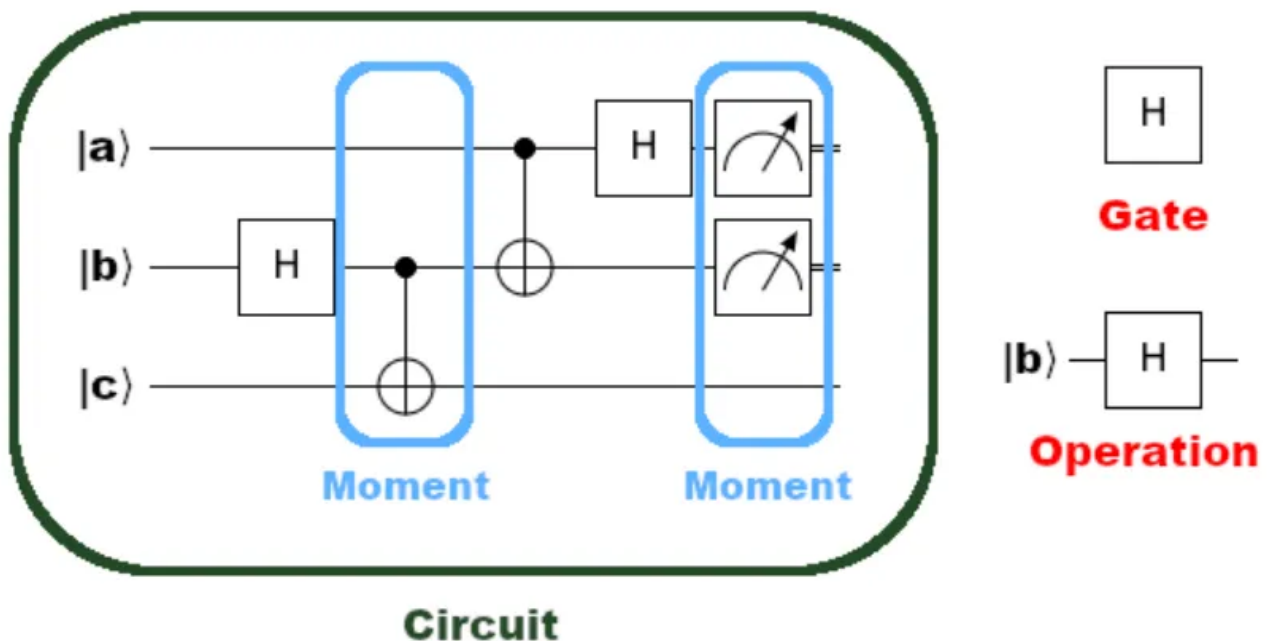
Furthermore, these notations will be operated using quantum logic gates. A qubit is the basic unit of quantum information, a quantum bit: a two-level system that can exist in a superposition of those two possible states.

GATES AND OPERATIONS

Quantum gates are unitary operators and are described as unitary matrices relative to some basis. In quantum computing and specifically the quantum circuit model of computation, a **quantum logic gate** (or simply **quantum gate**) is a basic quantum

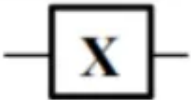
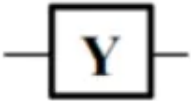
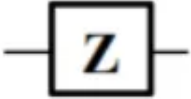
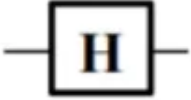
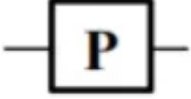
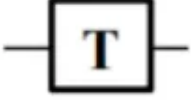
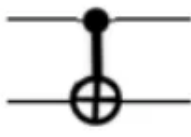
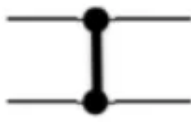
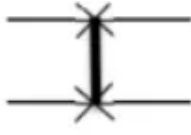
circuit operating on a small number of qubits. They are the building blocks of quantum circuits like classical logic gates are for conventional digital circuits. A Gate is an effect that can be applied to a collection of qubits (objects with a Qid). Gates can be applied to qubits by calling their on method, or, alternatively calling the gate on the qubits. The object created by such calls is an Operation.

Gates versus operations



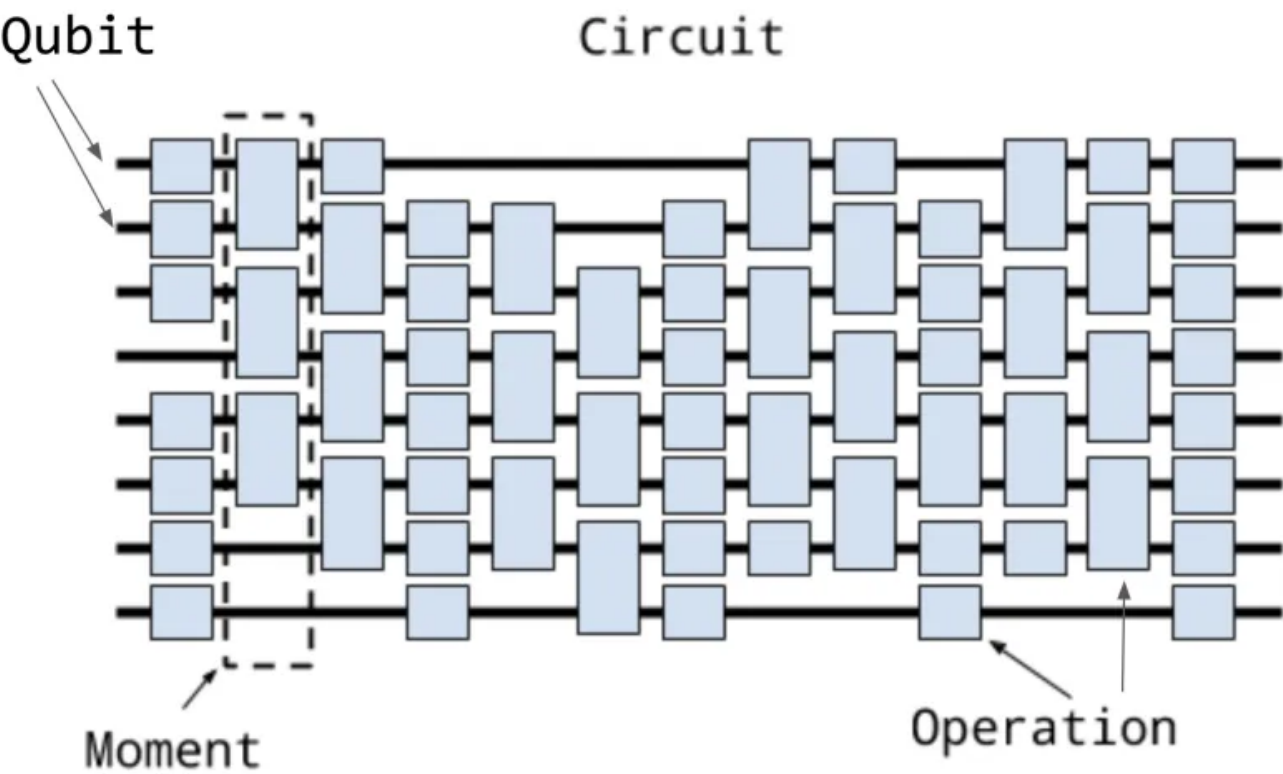
The above example shows the first half of a quantum teleportation circuit, found in many quantum computation textbooks. This example uses three different gates: a Hadamard (H), controlled-Not (CNOT), and measurement. These are represented in *cirq* by *cirq.H*, *cirq.CNOT*, and *cirq.measure*, respectively.

Quantum computing has its own logic gate, namely a quantum logic gate (quantum gate). There are several quantum logic gates, which are as follows:

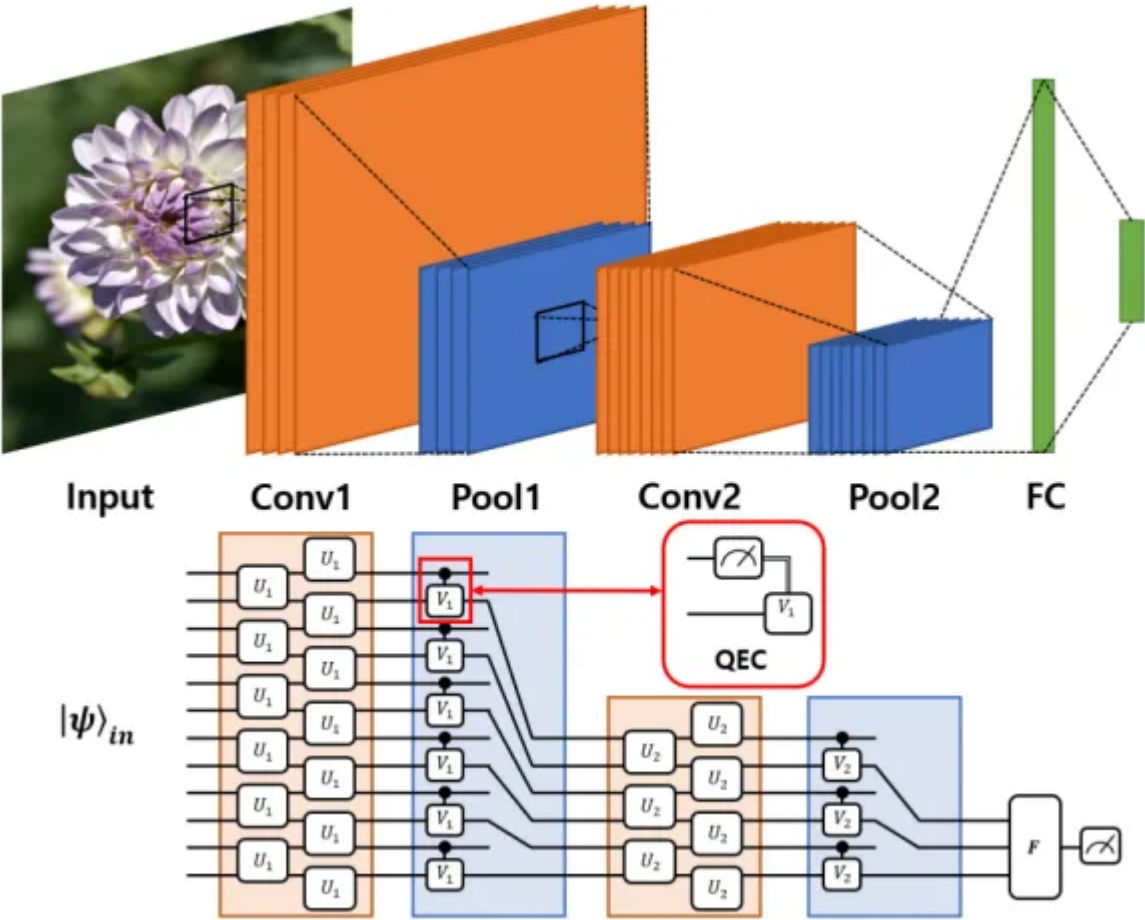
Operator	Matriks	Gerbang logika
Pauli-X (X)	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	
Pauli-Y (Y)	$\begin{bmatrix} 0 & i \\ i & 0 \end{bmatrix}$	
Pauli-Z (Z)	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	
Hadamard (H)	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	
Phase (S,P)	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	
$\pi/8$ (T)	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	
Controlled Not (CNOT, CX)	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	
Controlled Z (Z)	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$	
SWAP	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	

CIRCUIT

A Circuit is a collection of Moments. A Moment is a collection of Operations that all activities during the same abstract time slice. An Operation is some effect that operates on a specific subset of Qubits, the most common type of Operation is a GateOperation.



ARCHITECTURE CNN VS QCNN



Convolutional Neural Network (CNN) is a deep learning method used to process image data. In the process, CCN has several main layers, namely the Convolution

layer, Subsampling Layer, and fully connected layer.

Quantum Convolutional Neural Network (QCNN) is a development of CNN based on quantum computing. QCNN extends the main features and structures of CNNs into quantum systems (Oh et al. 2020).

BUILD QUANTUM CONVOLUTIONAL NEURAL NETWORK

INSTALL LIBRARY

For the manufacture of a quantum convolution neural network, it is recommended to use [google collab](#) or [Kaggle notebook](#), because both environments already support the TensorFlow-Quantum package.

```
!pip install -q tensorflow==2.4.1
!pip install -q tensorflow-quantum
```

Hosted on [Jovian](#)

[View File](#)

import package

```
import tensorflow as tf
import tensorflow_quantum as tfq

import cirq
import sympy
import numpy as np
import seaborn as sns
import collections

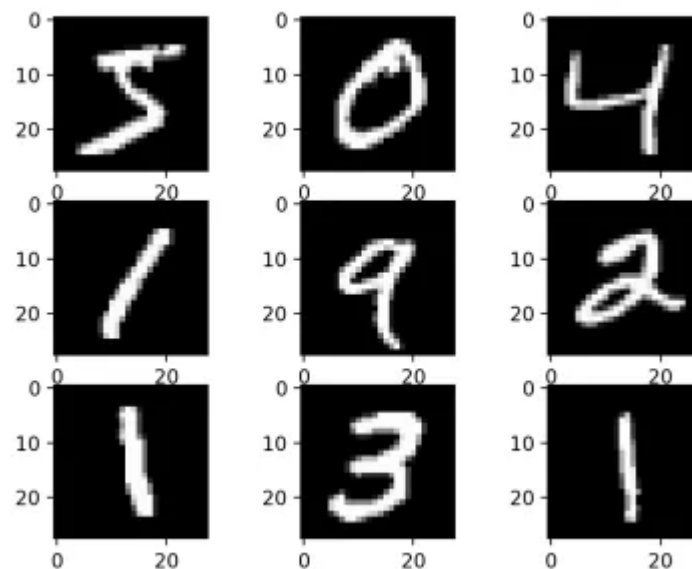
# visualization tools
%matplotlib inline
import matplotlib.pyplot as plt
from cirq.contrib.svg import SVGCircuit
```

Hosted on [Jovian](#)

[View File](#)

INPUT DATA MNIST

The data that will be used is the MNIST database (Modified National Institute of Standards and Technology database).



MNIST DATABASE

```
# load data mnist
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Rescale the images from [0,255] to the [0.0,1.0] range.
x_train, x_test = x_train[... , np.newaxis]/255.0, x_test[... , np.newaxis]/255.0

#cek length of data
print("Number of original training examples:", len(x_train))
print("Number of original test examples:", len(x_test))
print(x_train.shape, x_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11493376/11490434 [=====] - 0s 0us/step

Number of original training examples: 60000

Number of original test examples: 10000

(60000, 28, 28, 1) (10000, 28, 28, 1)

Hosted on [lovian](#)

[View File](#)

The MNIST data that will be used is only two-digit numbers 3 and 6.



Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

[View File](#)

example data:



Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

[View File](#)

In the process of implementing QCNN with large data, it still requires a large enough resource, therefore for this tutorial, we will resize data to 4 x 4 pixels, and remove contradicting data.



Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

[View File](#)

```
new_x = []  
new_y = []  
for flatten_x in mapping:
```

[Open in app](#)[Sign up](#)[Sign in](#)

```
else:  
    # Throw out images that match more than one label.  
    pass  
  
num_uniq_3 = sum(1 for value in mapping.values() if len(value) == 1)  
num_uniq_6 = sum(1 for value in mapping.values() if len(value) == 1)  
num_uniq_both = sum(1 for value in mapping.values() if len(value) == 2)  
  
print("Number of unique images:", len(mapping.values()))  
print("Number of unique 3s: ", num_uniq_3)  
print("Number of unique 6s: ", num_uniq_6)  
print("Number of unique contradicting labels (both 3 and 6): ", num_uniq_both)  
print()  
print("Initial number of images: ", len(xs))  
print("Remaining non-contradicting unique images: ", len(new_x))  
  
return np.array(new_x), np.array(new_y)
```

```
x_train_nocon, y_train_nocon = remove_contradicting(x_train_small, y_train_small)
```

Number of unique images: 10387

Number of unique 3s: 4912

Number of unique 6s: 5426

Number of unique contradicting labels (both 3 and 6): 49

Initial number of images: 12049

Remaining non-contradicting unique images: 10338

the next process is the binning process on pixel data with a limit value of 0.5, where a pixel value of more or equal to 0.5 will have a value of 1, and a pixel value of less than 0.5 will have a value of 0. One example of the transformation process above are as follows:

0.40	0.44	0.30	0.43
0.47	0.45	0.49	0.49
0.37	0.54	0.65	0.46
0.62	0.78	0.73	0.50



0	0	0	0
0	0	0	0
0	1	1	0
1	1	1	1

```
THRESHOLD = 0.5
```

```
x_train_bin = np.array(x_train_nocon > THRESHOLD, dtype=np.float32)  
x_test_bin = np.array(x_test_small > THRESHOLD, dtype=np.float32)
```

Hosted on [Jovian](#)

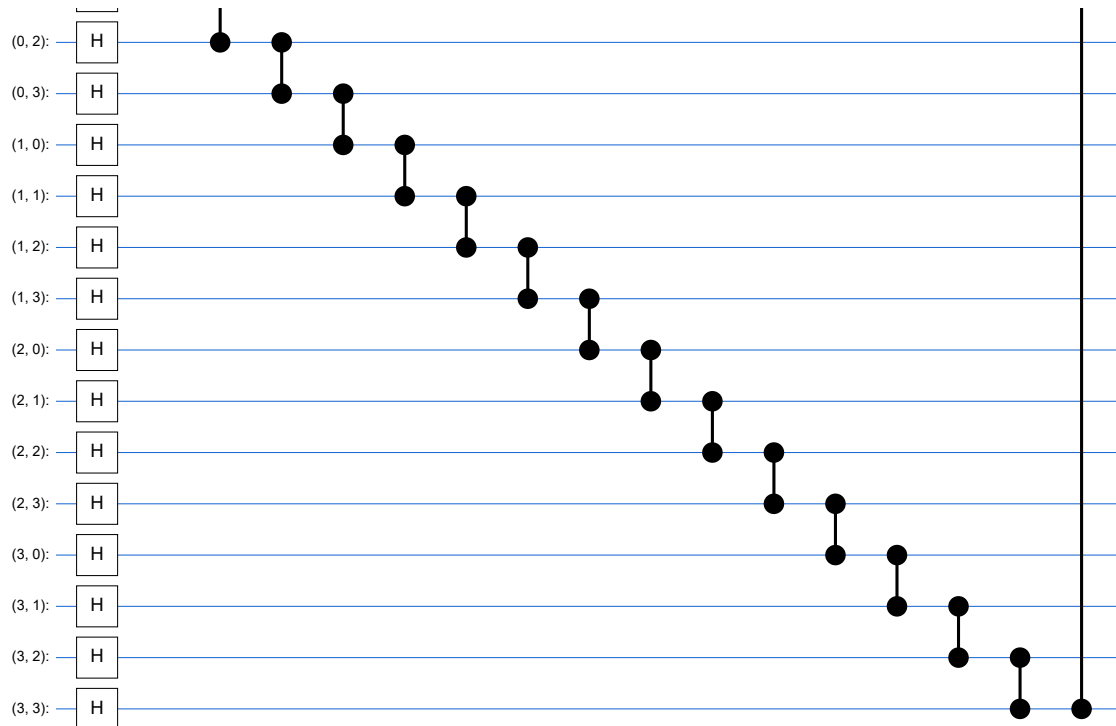
[View File](#)

The next stage is to prepare the cluster state as a data processing environment because the data used is 4 x 4 pixels, therefore it takes a cluster state with a total of 16 qubits, the cluster states are as follows:

```
def cluster_state_circuit(bits):
    """Return a cluster state on the qubits in `bits`."""
    circuit = cirq.Circuit()
    circuit.append(cirq.H.on_each(bits))
    for this_bit, next_bit in zip(bits, bits[1:] + [bits[0]]):
        circuit.append(cirq.CZ(this_bit, next_bit))
    return circuit
```

```
SVGCircuit(cluster_state_circuit(cirq.GridQubit.rect(4, 4)))
```

findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.



Hosted on [Jovian](#)

[View File](#)

The next process is to transform the data into a data circuit. In this transformation, the Pauli-x gate is added for data with a value of one, and for data with a value of zero, no Pauli-x gate is added.



Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

[View File](#)



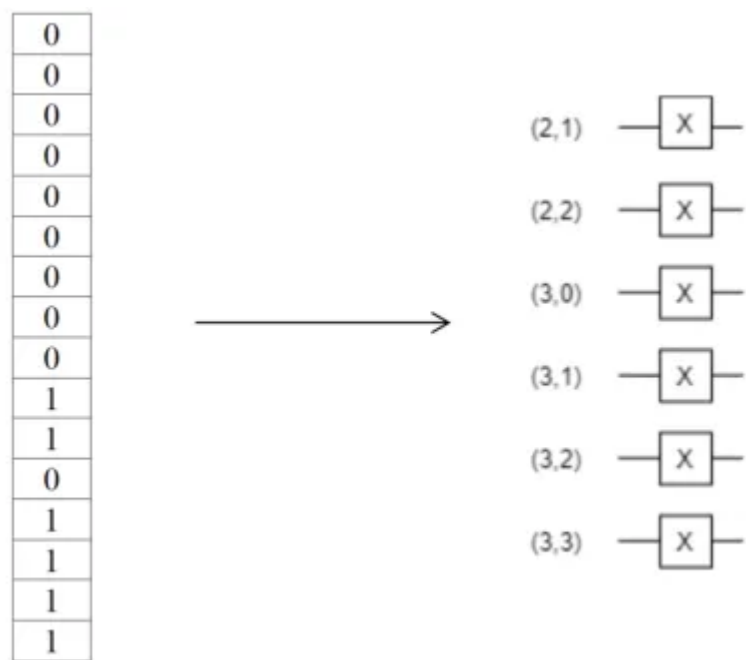
Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

[View File](#)

One example of the transformation above is as follows:



Unable to load content.
Please try again after some time.



Unable to load content.

Please try again after some time.

Hosted on [Jovian](#)

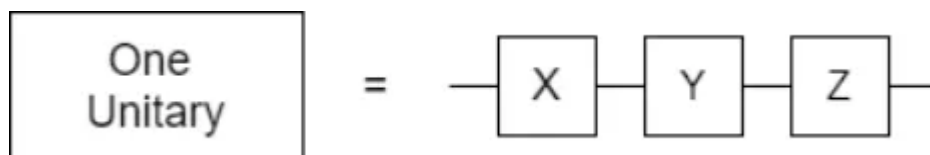
[View File](#)

BUILD ARSITEKTUR

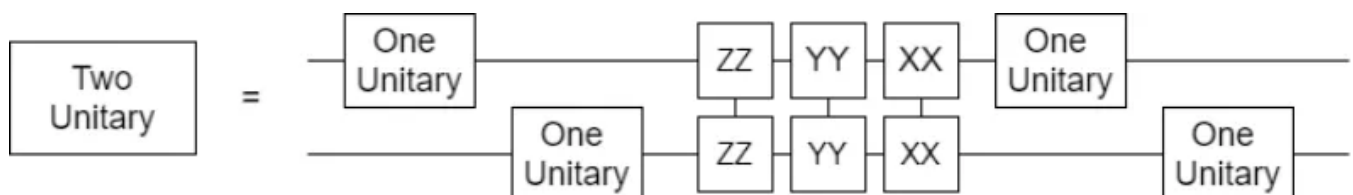
The making of the Quantum Convolutional Neural Network Architecture has several stages, namely the manufacture of **Quantum Circuits**, the construction of the **Quantum Layer**, and the construction of the **Quantum Convolutional Neural Network Architecture**.

Quantum Circuit

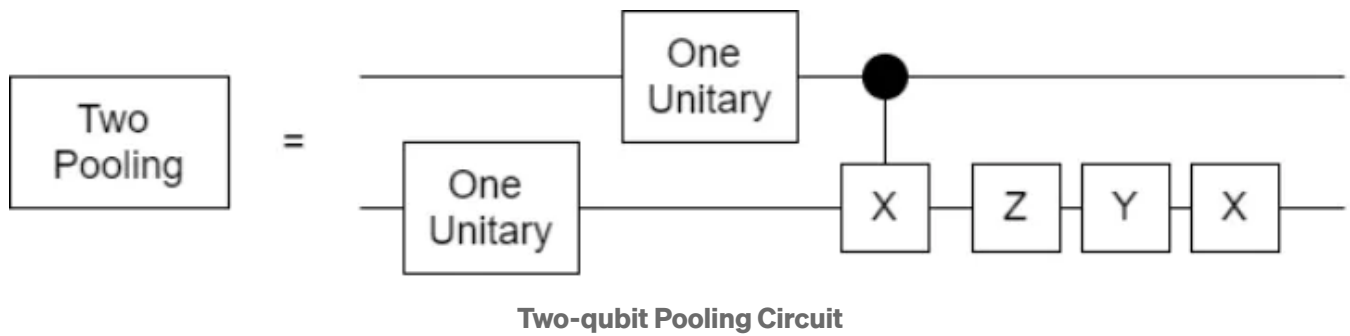
At the stage of forming a quantum circuit, three main quantum circuits are needed, namely *one qubit unitary circuit*, *two-qubit unitary circuit*, and *two-qubit pooling circuit*, in which the quantum circuit is as follows:



One Qubit Unitary Circuit



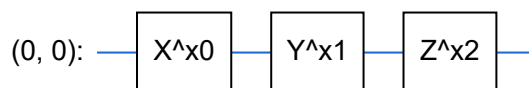
Two-qubit Unitary Circuit



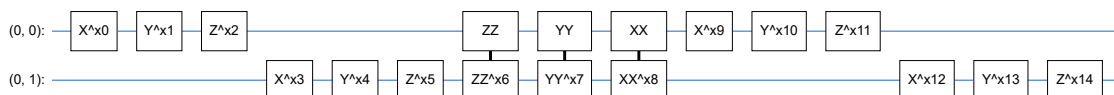
```
circuit += one_qubit_unitary(bits[0], symbols[9:12])
circuit += one_qubit_unitary(bits[1], symbols[12:])
return circuit
```

```
def two_qubit_pool(source_qubit, sink_qubit, symbols):
    """Make a Cirq circuit to do a parameterized 'pooling' operation, which
    attempts to reduce entanglement down from two qubits to just one."""
    pool_circuit = cirq.Circuit()
    sink_basis_selector = one_qubit_unitary(sink_qubit, symbols[0:3])
    source_basis_selector = one_qubit_unitary(source_qubit, symbols[3:6])
    pool_circuit.append(sink_basis_selector)
    pool_circuit.append(source_basis_selector)
    pool_circuit.append(cirq.CNOT(control=source_qubit, target=sink_qubit))
    pool_circuit.append(sink_basis_selector**-1)
    return pool_circuit
```

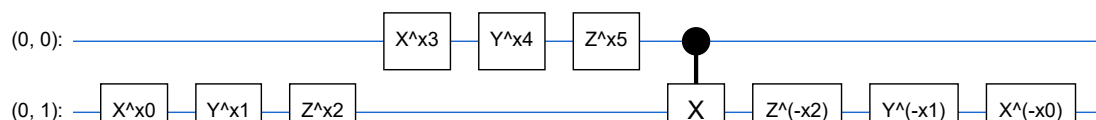
```
SVGCircuit(one_qubit_unitary(cirq.GridQubit(0, 0), sympy.symbols('x0:3')))
```



```
circuit(two_qubit_unitary(cirq.GridQubit.rect(1, 2), sympy.symbols('x0:15')))
```



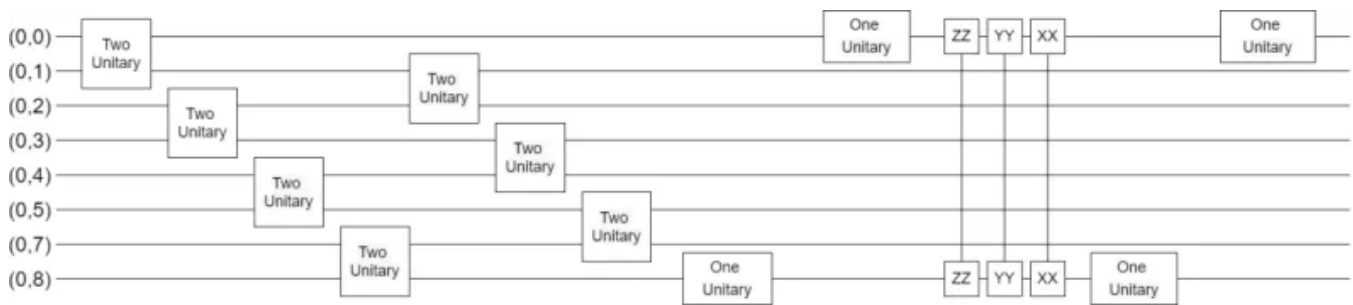
```
SVGCircuit(two_qubit_pool(*cirq.GridQubit.rect(1, 2), sympy.symbols('x0:6')))
```



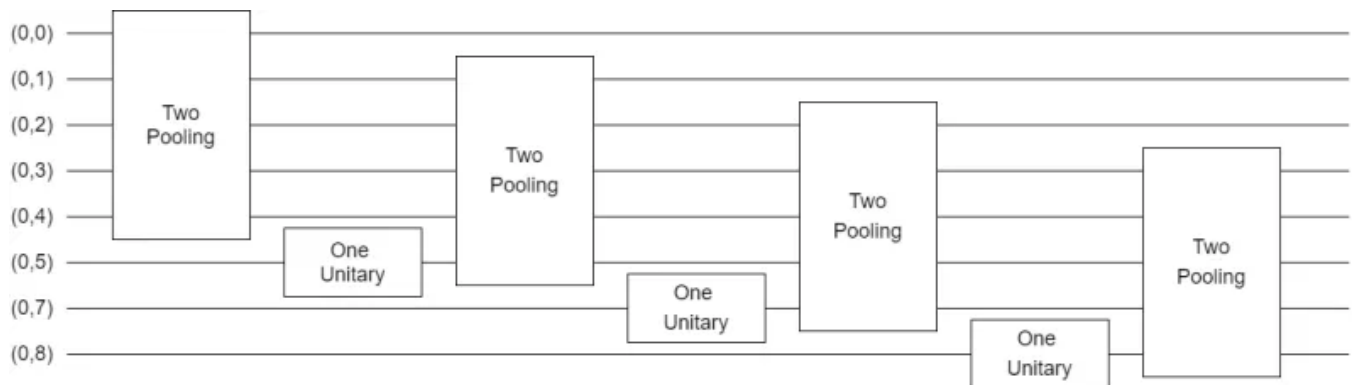
The circuit is a component to build a quantum layer architecture. In the next stage of development of the quantum layer.

Quantum Layer

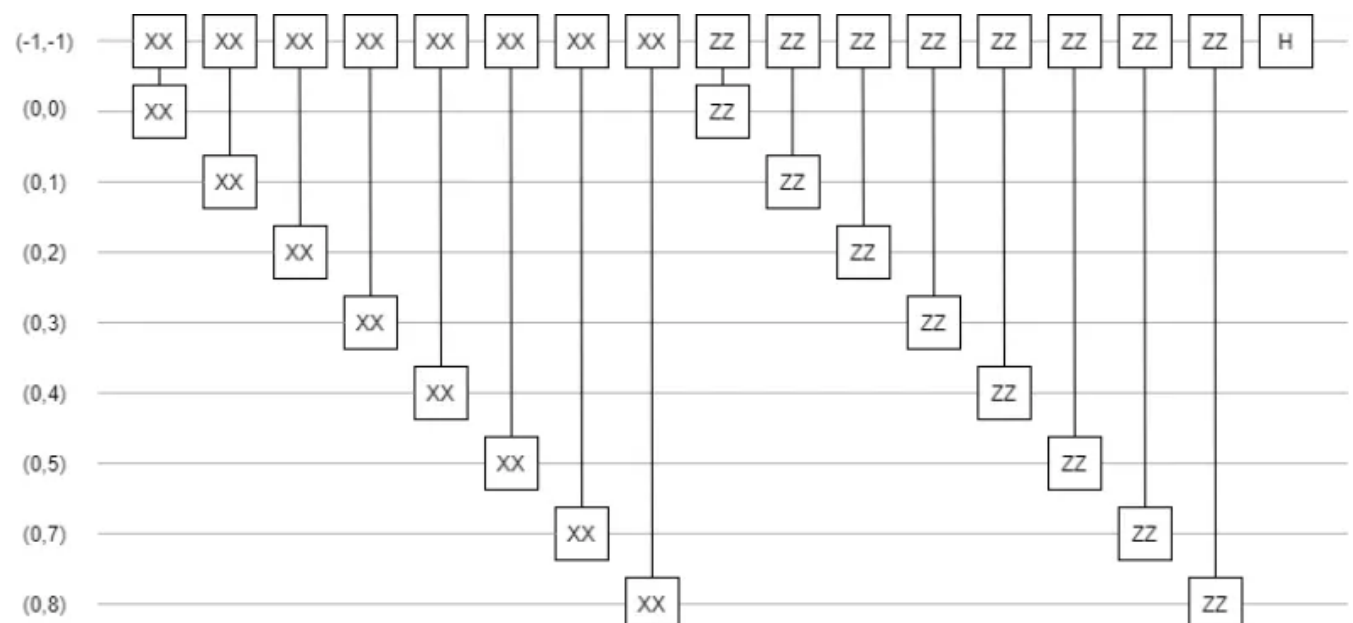
The construction of a quantum layer where there are three layers needed to build a Quantum Convolutional Neural Network Architecture, namely *quantum convolution layer*, *quantum pooling layer*, and *quantum neural network layer*, the layers are as follows:



Quantum Convolution Layer

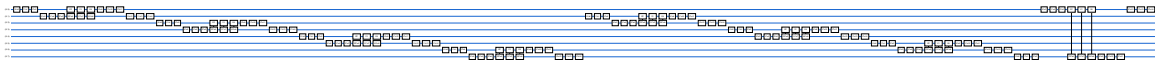


Quantum Pooling Layer



Quantum Neural Network Layer

```
SVGCircuit(
    quantum_conv_circuit(cirq.GridQubit.rect(1,8), sympy.symbols('x0:15'))
```



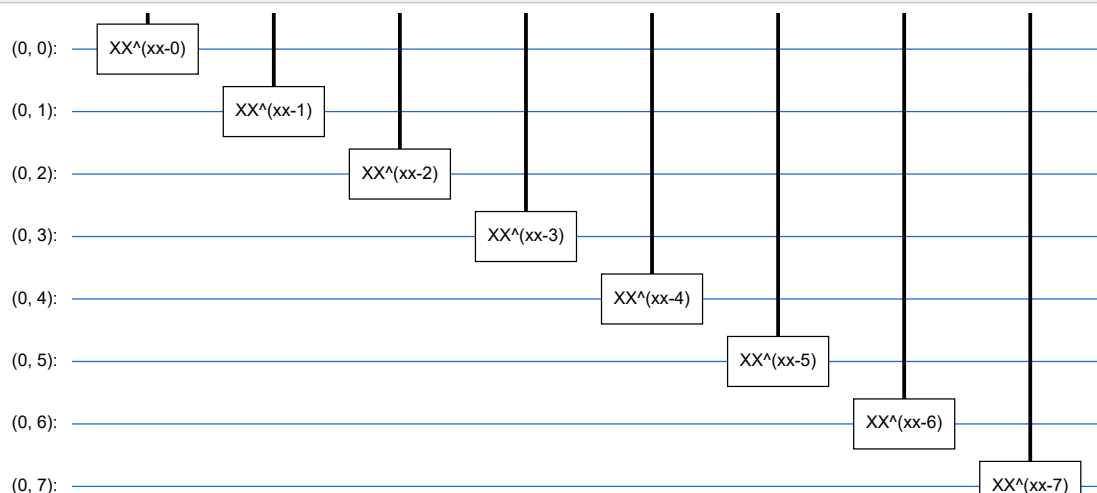
```
test_bits = cirq.GridQubit.rect(1, 8)
```

```
SVGCircuit(
    quantum_pool_circuit(test_bits[:4], test_bits[4:], sympy.symbols('x0:6'))
```



```
demo_builder = CircuitLayerBuilder(data_qubits = cirq.GridQubit.rect(1,8),
    readout=cirq.GridQubit(-1,-1))
```

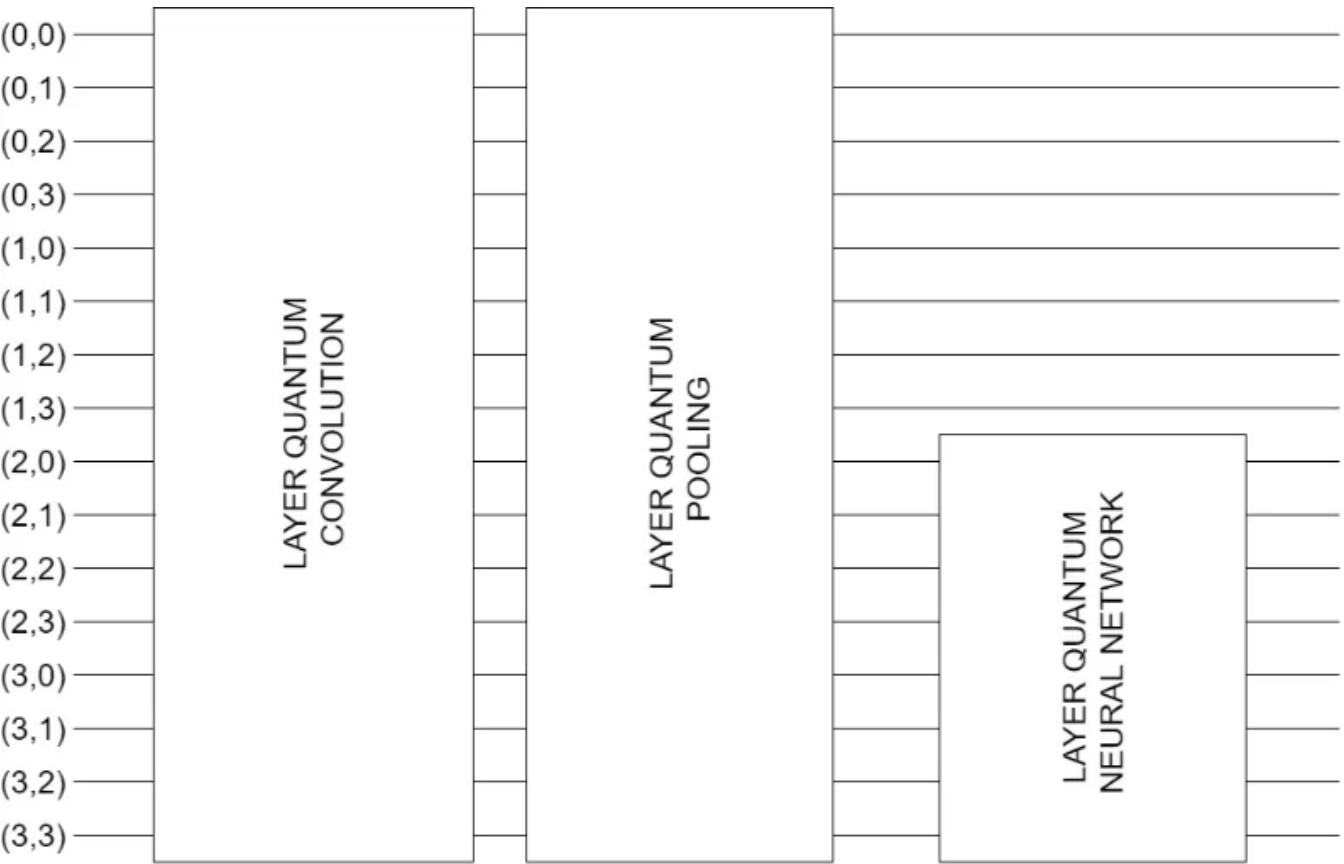
```
circuit = cirq.Circuit()
demo_builder.add_layer(circuit, gate = cirq.XX, prefix='xx')
SVGCircuit(circuit)
```



The quantum convolution layer consists of seven unitary two-qubit circuits and four one-qubit unitary circuits with an additional six quantum gates. The quantum pooling layer consists of four two-qubit pooling circuits and three one-qubit unitary circuits. Furthermore, the quantum neural network layer consists of an array of Pauli-x gates and Pauli-z gates. After the formation of the quantum layer, the next

step is to arrange the quantum layer into a Quantum Convolutional Neural Network Architecture.

Build Become QCNN Architecture



This Quantum Convolutional Neural Network architecture consists of a *quantum convolution layer* with 16 qubit data input, a *quantum pooling layer* with 16 qubit data input, and a *quantum neural network layer* with 8 qubit data input.

```

qubits = cirq.GridQubit.rect(4, 4)
readout = cirq.GridQubit(1,2)
circuit = cirq.Circuit()

symbols = sympy.symbols('qconv0:84')
circuit += quantum_conv_circuit(qubits , symbols[0:15]) # range symbols a
circuit += quantum_pool_circuit(qubits[:8], qubits[8:], symbols[15:21])

readout_qnn = cirq.GridQubit(1, 3)
data_qubits_qnn = cirq.GridQubit.rect(4, 4)[8:]

builder = CircuitLayerBuilder( data_qubits = data_qubits_qnn, readout=readout_qnn)

# Then add layers (experiment by adding more).
builder.add_layer(circuit, cirq.XX, "xx1")
builder.add_layer(circuit, cirq.ZZ, "zz1")

model_readout = circuit.append(cirq.H(cirq.GridQubit(1, 3)))

SVGCircuit(circuit)

```



```

readout_operators = cirq.Z(qubits[-1])

excitation_input = tf.keras.Input(shape=(), dtype=tf.dtypes.string)

cluster_state = tfq.layers.AddCircuit()(
    excitation_input, prepend=cluster_state_circuit(qubits))

quantum_model = tfq.layers.PQC(circuit,
                                readout_operators)(cluster_state)

qcnn_model = tf.keras.Model(inputs=[excitation_input], outputs=[quantum_model])

```



Training Model

for this tutorial, we will try simple modeling that doesn't take much time, with 5 epochs and using 500 data samples. If interested you can try it directly with the different parameters and how they change.

```
print(len(y_train_hinge), x_train_tfcirc.shape)
print(len(y_test_hinge), x_test_tfcirc.shape)
```

```
EPOCH = 5
BATCH_SIZE = 32
```

```
NUM_EXAMPLES = 500
#NUM_EXAMPLES = len(x_train_tfcirc)
```

```
NUM_EXAMPLES
```

```
10338 (10338,)
```

```
1968 (1968,)
```

```
500
```

```
x_train_tfcirc_sub = x_train_tfcirc[:NUM_EXAMPLES]
y_train_hinge_sub = y_train_hinge[:NUM_EXAMPLES]

print(x_train_tfcirc_sub.shape , y_train_hinge_sub.shape)
```

```
collections.Counter(y_train_hinge_sub)
```

```
(500,) (500,)
```

```
Counter({-1.0: 270, 1.0: 230})
```

Hosted on [Jovian](#)[View File](#)

```
def hinge_accuracy(y_true, y_pred):  
    y_true = tf.squeeze(y_true) > 0.0  
    y_pred = tf.squeeze(y_pred) > 0.0  
    result = tf.cast(y_true == y_pred, tf.float32)  
    return tf.reduce_mean(result)  
  
qcnv_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.02),  
                  loss=tf.losses.mse,  
                  metrics=[hinge_accuracy])  
  
history = qcnv_model.fit(x=x_train_tfcirc_sub,  
                        y=y_train_hinge_sub,  
                        batch_size=16,  
                        epochs=EPOCH,  
                        verbose=1,  
                        validation_data=(x_test_tfcirc , y_test_hinge))
```

Epoch 1/5

32/32 [=====] - 119s 4s/step - loss: 0.5943 -
hinge_accuracy: 0.7622 - val_loss: 0.4560 - val_hinge_accuracy: 0.7698

Epoch 2/5

32/32 [=====] - 120s 4s/step - loss: 0.4482 -
hinge_accuracy: 0.7833 - val_loss: 0.4947 - val_hinge_accuracy: 0.8333

Epoch 3/5

32/32 [=====] - 121s 4s/step - loss: 0.4780 -
hinge_accuracy: 0.8148 - val_loss: 0.4301 - val_hinge_accuracy: 0.8333

Epoch 4/5

32/32 [=====] - 122s 4s/step - loss: 0.4872 -
hinge_accuracy: 0.7852 - val_loss: 0.4350 - val_hinge_accuracy: 0.8206

Epoch 5/5

32/32 [=====] - 119s 4s/step - loss: 0.4638 -
hinge_accuracy: 0.7794 - val_loss: 0.4281 - val_hinge_accuracy: 0.8333

Hosted on [Jovian](#)

[View File](#)


```
0.46568000316619873,
0.4690588712692261,
0.4558601677417755,
0.4525804817676544],
'val_hinge_accuracy': [0.769817054271698,
0.8333333134651184,
0.8333333134651184,
0.8206300735473633,
0.8333333134651184],
'val_loss': [0.45602595806121826,
0.4946832060813904,
0.4301346242427826,
0.4350111782550812,
0.4281463027000427]}
```

```
import pandas as pd
df_history = pd.DataFrame()
df_history['accuracy'] = history.history['accuracy']
df_history['loss'] = history.history['loss']
df_history['val_accuracy'] = history.history['val_accuracy']
df_history['val_loss'] = history.history['val_loss']

df_history.to_csv("history.csv", index = False)

print(df_history.shape)
df_history.head()
```

(5, 4)

	accuracy	loss	val_accuracy	val_loss
0	0.789062	0.503847	0.769817	0.456026
1	0.775391	0.465680	0.833333	0.494683
2	0.812500	0.469059	0.833333	0.430135
3	0.785156	0.455860	0.820630	0.435011
4	0.773438	0.452580	0.833333	0.428146

The results of the modeling show quite good results, with the latest validation accuracy of 83.33.

This QCNN modeling still has many shortcomings and weaknesses, especially from the availability of the quantum computing environment that is not widely available, and we create the quantum environment in a classical computer, this is what causes quantum processing on a classical computer to be relatively long. I hope this paper can help readers with further research, and the development of quantum

computing. I know there are still many mistakes in my writing, I hope to get feedback and suggestions.

Thank you very much for reading this article.

References:

- <https://www.tensorflow.org/quantum>
- <https://quantumai.google/cirq>
- https://en.wikipedia.org/wiki/Quantum_logic_gate
- Watabe M, Shiba K, Chen C, Sogabe M, Sakamoto K, Sogabe T. 2021. Quantum circuit learning with error backpropagation algorithm and experimental implementation. *Journal Quantum Rep.* 3(2):333–349. doi.org/10.3390/quantum3020021.
- Gultom. 2017. Klasifikasi data dengan quantum perceptron. *Jurnal Teknovasi.* 4(1):1–9.
- Kaye P, Laflamme R, Mosca M. 2007. *An Introduction to Quantum Computing.* New York: Oxford University Press.
- Gado M, Younes A. 2021. Optimization of reversible circuit using toffoli decompositions with negative controls. *Jurnal Symmetry.* 13(6): 1025. doi.org/10.3390/sym13061025.
- Oh S, Choi J, Kim J. 2020. A tutorial on quantum Convolutional neural network (QCNN). *forthcoming.*

Quantum Computing

Machine Learning

Artificial Intelligence

Image Processing

Quantum Physics



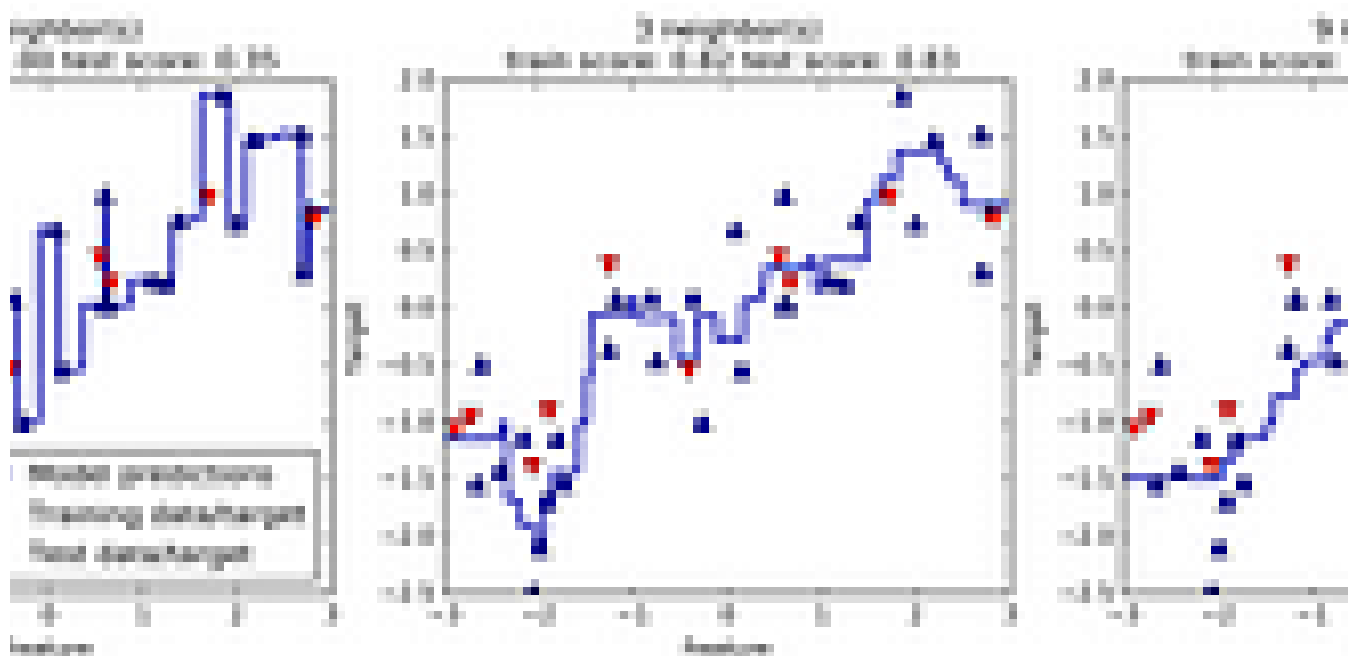
Follow

Written by Imam Muhajir

201 Followers · Writer for Analytics Vidhya

Data Scientist at [KECILIN.ID](https://www.kecilin.id) || Physicist || Writer about Data Analysis, Big Data, Machine Learning, and AI.
 LinkedIn: <https://www.linkedin.com/in/imammuhajir92/>

More from Imam Muhajir and Analytics Vidhya



Imam Muhajir in Analytics Vidhya

K-Neighbors Regression Analysis in Python

K nearest neighbors is a simple algorithm that stores all available cases and predict the numerical target based on a similarity measure...

3 min read · Apr 20, 2019



141





Kia Eisinga in Analytics Vidhya

How to create a Python library

Ever wanted to create a Python library, albeit for your team at work or for some open source project online? In this blog you will learn...

7 min read · Jan 27, 2020



2.2K



27



Harikrishnan N B in Analytics Vidhya

Confusion Matrix, Accuracy, Precision, Recall, F1 Score

Binary Classification Metric

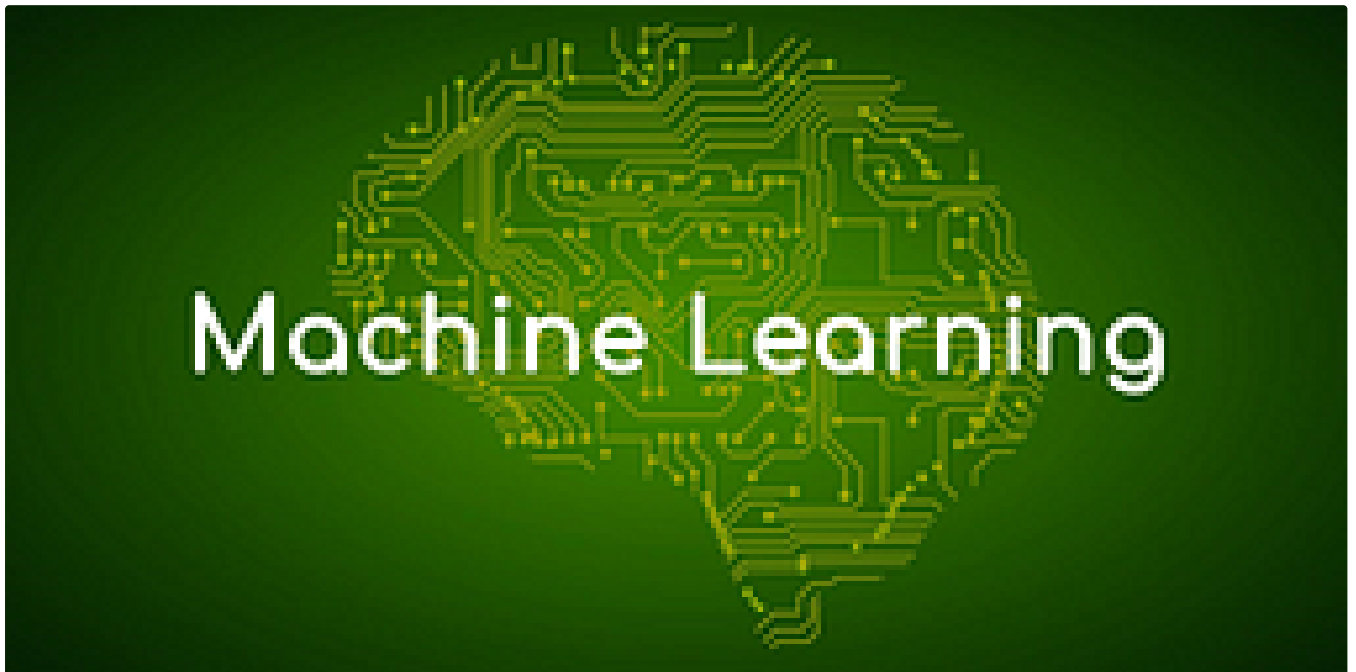
6 min read · Dec 10, 2019



896



6



Imam Muhajir in Analytics Vidhya

Ridge Regression with Multicollinearity in Python

Definition Ridge Regression

8 min read · Apr 27, 2019



109



See all from Imam Muhajir

See all from Analytics Vidhya

Recommended from Medium



Holly Emblem in Towards Data Science

Quantum Deep Learning: A Quick Guide to Quantum Convolutional Neural Networks

Everything you need to know about quantum convolutional neural networks (QCNNs), including the benefits and limitations of these...

9 min read · Oct 4, 2022

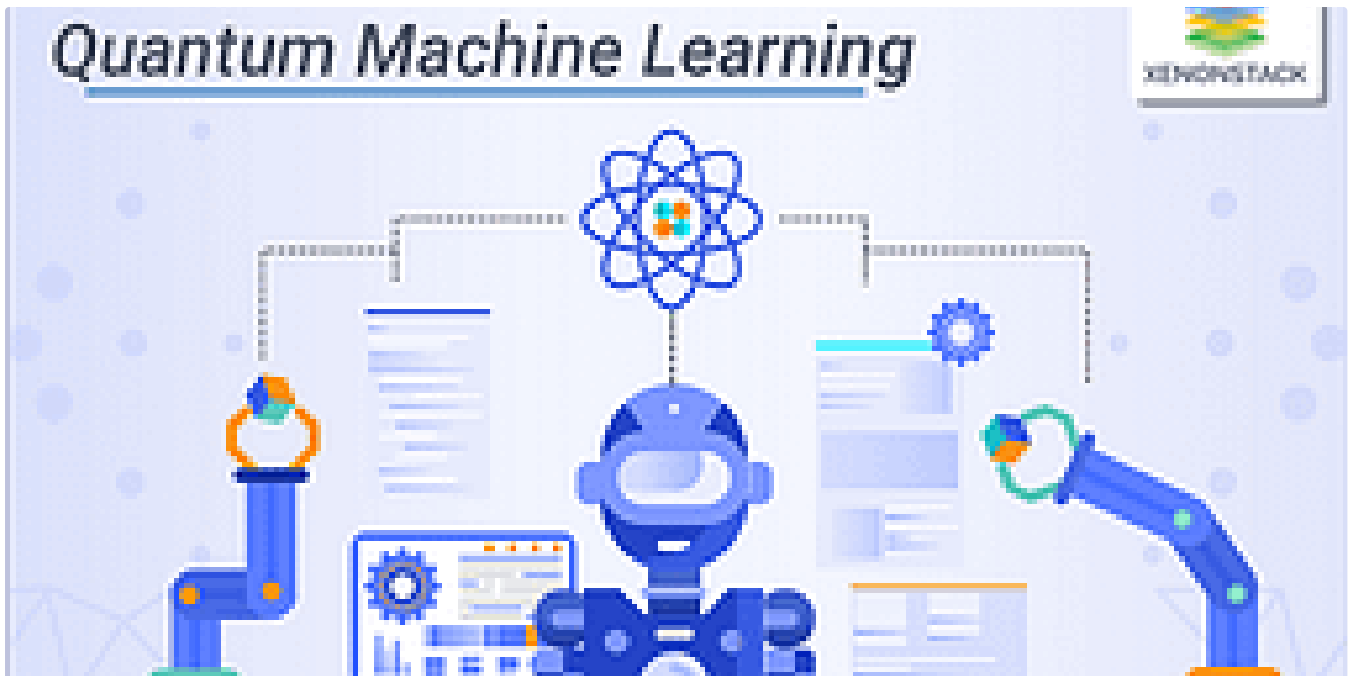


310



1





Suman Kumar Roy

Unleashing Quantum Capabilities: Navigating Classical and Quantum Machine Learning

Welcome to our Quantum Computing (QC) journey! In our last session, we delved into the foundational aspects of QC, exploring its...

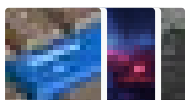
7 min read · Aug 2, 2023



9

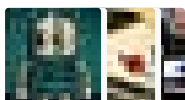


Lists



Predictive Modeling w/ Python

20 stories · 829 saves



AI Regulation

6 stories · 288 saves



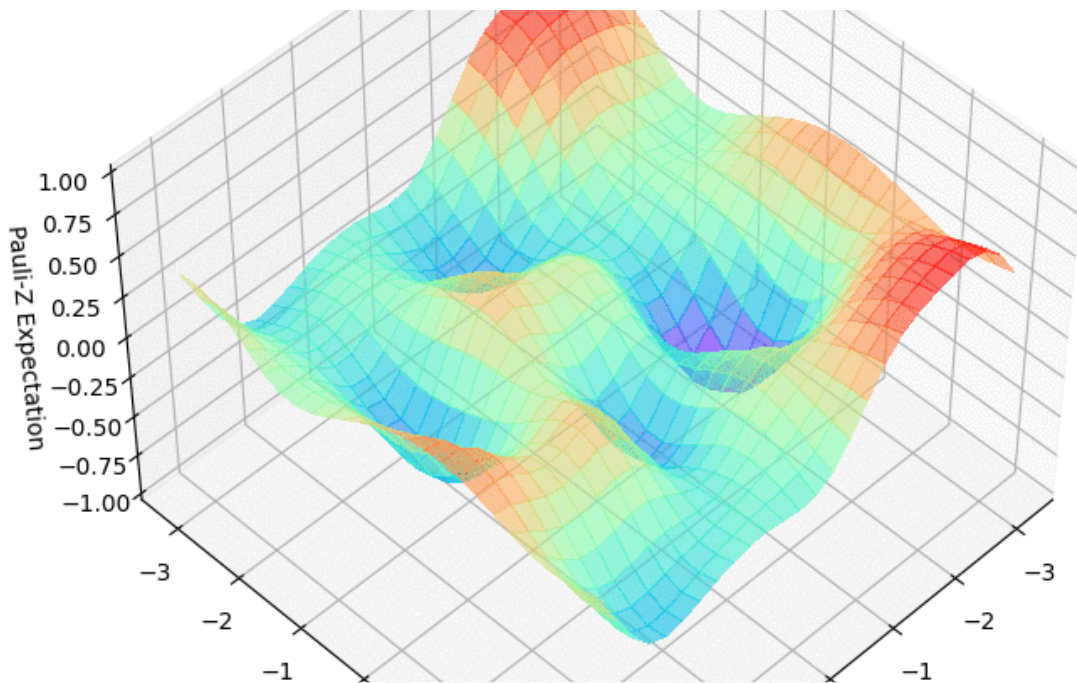
Natural Language Processing

1122 stories · 591 saves



Practical Guides to Machine Learning

10 stories · 956 saves



Gopal Ramesh Dahale in Qiskit

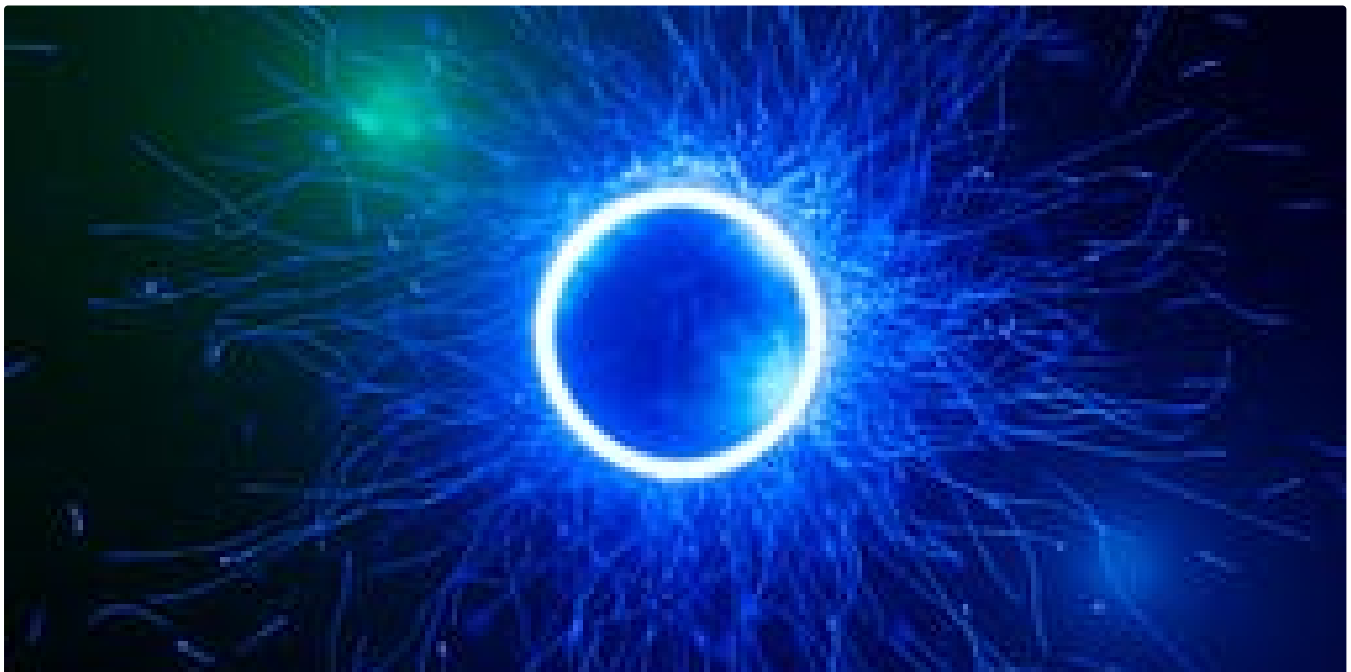
Layerwise learning for Quantum Neural Networks with Qiskit

By Gopal Ramesh Dahale

6 min read · Sep 6, 2023



87

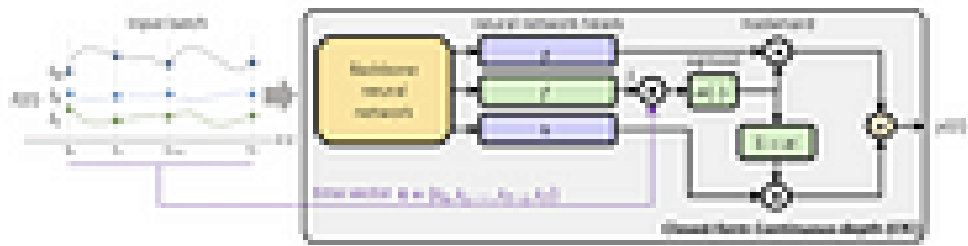


Sheroy Cooper

Quantum Superposition—Explained Simply and In-Depth

For the Curious Child in You and the Inquisitive Adult

2 min read · Sep 5, 2023



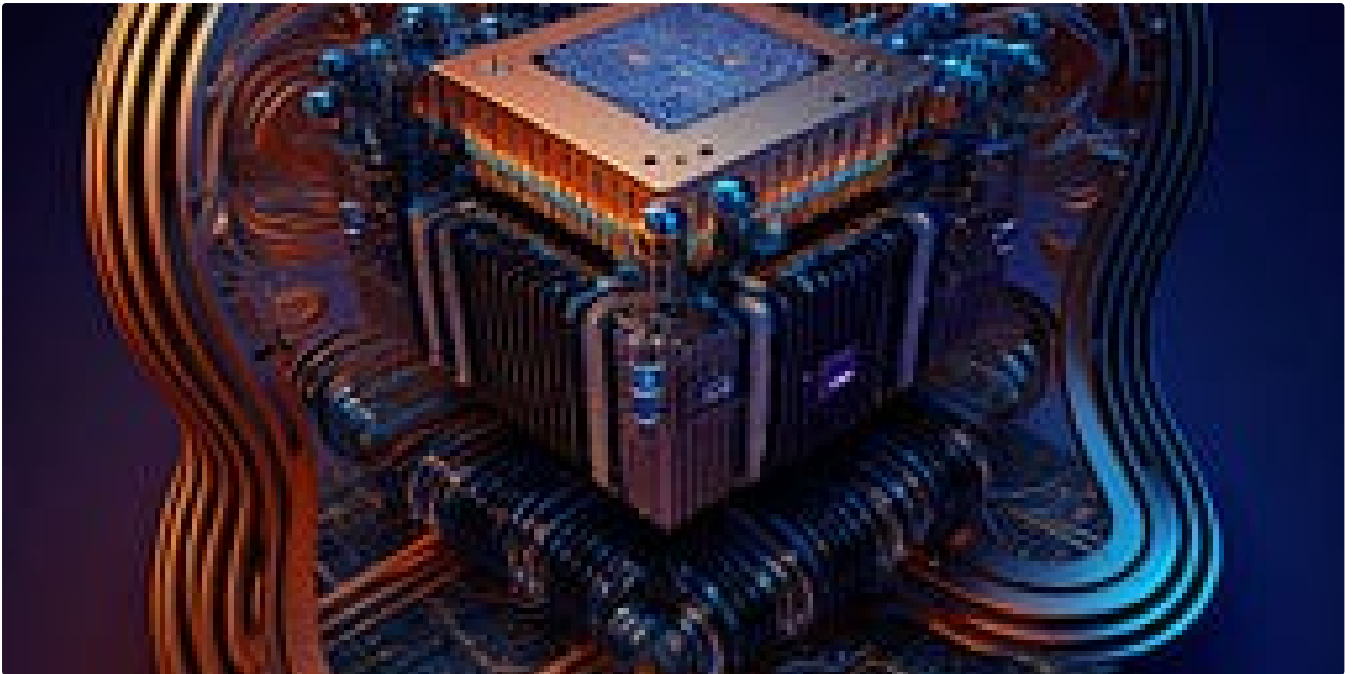
 fg-research

Time series forecasting with the CfC SageMaker Algorithm from AWS Marketplace

The closed-form continuous-depth network (CfC) is a new neural network architecture for sequential data [1]. CfCs belong to the class of...

5 min read · Sep 9, 2023





 David Jacob

Quantum Computing: Unlocking the Power of Quantum Bits

In the vast realm of technological innovation, quantum computing emerges as a revolutionary force, poised to redefine the limits of...

★ · 3 min read · 1 day ago



600



12



See more recommendations